



TAREA 03

Desarrollo y fase de pruebas

DESCRIPCIÓN BREVE

En esta tarea se llevará a cabo la implementación del proyecto, así como el desarrollo de la fase de pruebas.

Juan Manuel Muñoz Moreno

<https://github.com/JMMUNOZ-DAM/Pl.git>

Contenido

1. Implementación - Proyecto NETFIX.....	2
1.1 Aspectos Destacados del Desarrollo	2
1.1.1 Interfaz Moderna con FlatLaf:	2
1.1.2 Arquitectura MVC Modificada:.....	2
1.1.3 Simulación de Diagnóstico en Tiempo Real:	2
1.1.4 Seguridad Básica:	2
1.2. Problemas Encontrados y Soluciones.....	2
1.2.1 Gestión de la navegación entre pantallas:	2
1.2.2 Rigidez del Diseñador Gráfico de NetBeans:.....	3
1.2.3 Conexión a Base de Datos y "Leaks":	3
1.2.4 Conflictos con Git:	3
1.3. Trabajo Futuro y Mejoras	3
2. Documentación de Pruebas Unitarias y de Integración: TestUnilIntegra.....	4
2.1 Pruebas de Unidad (Lógica de Negocio).....	4
2.2 Pruebas de Integración (Base de Datos)	6
2.3 Interpretación de Resultados	6
2.4 Se muestra el código de la clase "TestUnilIntegra "	7
3. Respuestas de Gestión y Planificación del Proyecto	9
3.1 ¿Ha podido cumplir la planificación? ¿Qué problemas ha tenido?	9
3.2 Principales problemas y ajustes:	9
3.3 ¿Su proyecto necesita algún tipo de permiso o autorización administrativa?.....	9
3.4 ¿Ha establecido algún documento de prevención de riesgos laborales?	9
3.5 Valoración económica respecto a su ejecución.....	10
4. Guía de instalación/Configuración.	11
4.1 Configuración Base de datos.	11
4.2 Instalación de la aplicación.....	13
5. Usuarios para realizar las pruebas.	15

1. Implementación- Proyecto NETFIX

Esta parte del documento recoge los puntos clave del desarrollo de la aplicación NETFIX, destacando las decisiones tomadas, los problemas que me he ido encontrando y cómo los he resuelto, así como las ideas que tengo para futuras versiones.

1.1 Aspectos Destacados del Desarrollo

Lo que más me ha gustado implementar o lo que considero más interesante del proyecto:

1.1.1 Interfaz Moderna con FlatLaf:

No quería que la aplicación tuviera el típico aspecto gris y antiguo de Swing/Java. He dedicado tiempo a integrar la librería "FlatLaf" para darle un toque limpio y moderno. Me ha permitido unificar colores, bordes redondeados y fuentes en toda la aplicación de forma "centralizada" usando una clase `ThemeManager`. De esta forma, si quiero cambiar el color principal de la app, solo lo cambio en un sitio.

1.1.2 Arquitectura MVC Modificada:

He intentado seguir el patrón Modelo-Vista-Controlador para mantener el código ordenado. Tengo paquetes separados para la lógica (`controlador`), las ventanas (`vista`) y los datos (`modelo`). Aunque al principio cuesta más estructurarlo, luego facilita mucho saber dónde buscar si falla algo en la base de datos o en un botón.

1.1.3 Simulación de Diagnóstico en Tiempo Real:

Para la parte de los técnicos, implementé un pequeño simulador (`SimuladorDiagnostico`) que genera datos aleatorios de red (velocidad, latencia, etc.) y usa hilos (`SwingWorker`) para no congelar la pantalla, dando la sensación de que está "midiendo" de verdad.

1.1.4 Seguridad Básica:

No guardo las contraseñas en texto plano en la base de datos. Utilicé BCrypt para hashearlas. Me pareció un detalle importante de seguridad, es simple pero efectivo.

1.2. Problemas Encontrados y Soluciones

Durante el desarrollo me surgieron varios bloqueos que tuve que solucionar:

1.2.1 Gestión de la navegación entre pantallas:

- * **El problema:** Al principio abría muchas ventanas (`New JFrame`) y era un lío para el usuario.
- * **La solución:** Implementé un `MainFrame` principal con un diseño de CardLayout. Así, el menú lateral siempre está fijo y solo cambia el panel central. Da una experiencia mucho más fluida, como una web o app moderna.

1.2.2 Rigidez del Diseñador Gráfico de NetBeans:

* **El problema:** El asistente gráfico de NetBeans es muy visual, pero genera bloques de código "protegidos" (en gris/azul) que no te deja tocar. Eso me frustraba mucho cuando quería meter diseños dinámicos, bucles para botones o simplemente retocar un layout de forma precisa, ya que el IDE forzaba su propio código intocable.

* **La solución:** Tomé la decisión de abrir y editar las clases de la vista directamente con **VSCode**. Al hacerlo "a mano", gané libertad total para limpiar código redundante, reorganizar los componentes y aplicar mis propios estilos sin que NetBeans me bloqueara. Fue un poco arriesgado mezclar editores, pero valió la pena para tener el control real del código fuente de la interfaz.

1.2.3 Conexión a Base de Datos y "Leaks":

* **El problema:** A veces, si hacía muchas consultas, la aplicación fallaba o no finalizaba las conexiones.

* **La solución:** Creé una clase `DatabaseManager` para reutilizar la conexión. También revisé el código para asegurarme de cerrar siempre los `ResultSet` y `PreparedStatement` dentro de bloques `try-catch` o `finally`.

1.2.4 Conflictos con Git:

* **El problema:** Al trabajar en diferentes ramas (`desarrollo`, `entrega`) y hacer pruebas, a veces tenía conflictos al hacer merge o push, sobre todo con archivos que cambiaba localmente.

* **La solución:** Decidí mantener un flujo de trabajo limpio: trabajar en una rama aparte y solo subir a la principal cuando el código funcionase seguro. En caso de desastre (como cuando intenté meter el tema oscuro y se rompió), aprendí a usar `git reset --hard` para volver a una versión estable conocida.

1.3. Trabajo Futuro y Mejoras

* Modo Oscuro Real:

Intenté implementarlo (un botón para cambiar entre claro/oscuro en tiempo real), pero daba problemas de refresco en algunos componentes y afectaba a la estabilidad. Decidí dejarlo fuera de esta entrega para priorizar que la app funcione perfecta, pero es lo primero que añadiría.

* Generación de PDFs:

Sería muy útil que los técnicos pudieran exportar el "parte de incidencia" o el "resultado del diagnóstico" a un archivo PDF directamente desde la app para enviárselo al cliente.

* Versión Web/Móvil:

Ahora mismo es una app de escritorio (Java Swing). En el futuro, lo ideal sería migrar el backend a una API REST (Spring Boot, por ejemplo) y hacer un frontend web para que se pueda acceder desde cualquier sitio sin instalar nada.

* Chat en tiempo real:

Estaría bien poder abrir un chat entre compañeros, por ejemplo, poder consultar directamente a tu supervisor si te surgiera alguna duda con una incidencia.

2. Documentación de Pruebas Unitarias y de Integración: TestUnilIntegra.

Esta parte del documento detalla la batería de pruebas implementada en la clase TestUnilIntegra. Esta clase funciona como un ejecutor de pruebas independiente que verifica tanto la lógica de negocio (Pruebas Unitarias) como la conectividad con la base de datos (Pruebas de Integración).

Resumen General

Clase Principal: TestUnilIntegra.java

Ubicación: src/main/java/com/jmmunoz/netfix/tests/

Objetivo: Validar componentes críticos sin necesidad de levantar toda la interfaz gráfica.

Método de Ejecución: Ejecutar el método main de esta clase.

2.1 Pruebas de Unidad (Lógica de Negocio)

Estas pruebas se ejecutan sin conexión a la base de datos y verifican el correcto funcionamiento de métodos utilitarios y modelos de datos.

ID	Nombre de la Prueba	Descripción	Resultado
U01	Email Valid (.com)	Verifica que el validador de correos acepte dominios .com.	true (Válido)
U02	Email Valid (.es)	Verifica que el validador de correos acepte dominios .es.	true (Válido)
U03	Email Invalid (gmail)	Verifica que se rechacen dominios externos no corporativos (ej. gmail.com).	false (Inválido)
U04	Hashing no vacío	Prueba que la función de hashing de contraseñas (hashPass) genere una cadena no vacía, y distinta al texto original.	true
U05	Verificación BCrypt	Valida que una contraseña en texto plano coincida con su hash generado anteriormente usando BCrypt.	true (Coinciden)
U06	Actualización de Contraseña	Verifica que al solicitar un cambio de contraseña, se genere un nuevo hash diferente al anterior.	true (Hash cambia)

ID	Nombre de la Prueba	Descripción	Resultado
U07	Pass Update (Vacío)	Verifica que, si se intenta actualizar la contraseña con un campo vacío, se mantenga el hash anterior (no se modifique).	true (Mantiene hash)
U08	Modelo Usuario Props	Prueba la creación de un objeto Usuario, verificando que el constructor y los <i>getters</i> asignen y retornen los valores correctamente.	true
U09	Simulador Instanciación	Verifica que la clase SimuladorDiagnostico pueda ser instanciada sin errores.	true (No nulo)
U10	Verificación Booleana	Prueba de control básica para asegurar que el sistema de aserciones del test runner funciona.	true

2.2 Pruebas de Integración (Base de Datos)

Estas pruebas requieren que el servidor de base de datos MySQL esté activo y accesible según la configuración en AppConfig.

ID	Nombre de la Prueba	Descripción	Resultado Esperado
I01	Conexión DB (Roles)	Intenta establecer una conexión real a la base de datos y ejecutar una consulta SELECT sobre la tabla de perfiles/roles.	true (Conexión exitosa y datos recuperados)

2.3 Interpretación de Resultados

Al ejecutar la clase, la consola mostrará un reporte detallado:

- **[OK]:** La prueba pasó exitosamente.
- **[FAIL]:** La prueba falló. Se mostrará el valor esperado vs. el obtenido.

Al final del log se presenta un resumen con el total de pruebas ejecutadas, pasadas y fallidas.

2.4 Se muestra el código de la clase “TestUnilIntegra”.

```
public class TestUnilIntegra {

    // Contadores
    private static int testsRun = 0;
    private static int testsPassed = 0;
    private static int testsFailed = 0;

    Run | Debug
    public static void main(String[] args) {
        System.out.println("=====");
        System.out.println(" INICIANDO BATERÍA DE PRUEBAS NETFIX ");
        System.out.println("=====");

        // --- PRUEBAS DE UNIDAD ---
        System.out.println("\n--- [1] PRUEBAS DE UNIDAD (LOGICA) ---");

        // U01 - U03: Email
        Utilities ut = new Utilities();
        test("U01", "Email Valid (.com)",
            "Verificando que un email con dominio .com sea aceptado.",
            "\"test@netfix.com\"",
            ut.checkEmail("test@netfix.com"), true);

        test("U02", "Email Valid (.es)",
            "Verificando que un email con dominio .es sea aceptado.",
            "\"test@netfix.es\"",
            ut.checkEmail("test@netfix.es"), true);

        test("U03", "Email Invalid (gmail)",
            "Verificando que un email con dominio externo (gmail) sea rechazado.",
            "\"test@gmail.com\"",
            ut.checkEmail("test@gmail.com"), false);

        // U04: Generación de Hash
        String hash = ut.hashPass("secret");
        test("U04", "Hashing no vacío",
            "Verificando que hashPass genere una cadena no vacía y diferente al input.",
            "\"secret\"",
            hash != null && !hash.isEmpty() && !hash.equals("secret"), true);

        // U05: Verificación BCrypt
        boolean isMatch = Utilities.PasswordUtils.checkPassword("secret", hash);
        test("U05", "Verificación BCrypt",
            "Verificando que BCrypt valide correctamente el password original contra su hash.",
            "\"secret\", \"" + hash + "\"",
            isMatch, true);

        // U06 - U07: Lógica de Actualización de Contraseña
        String newHash = ut.procesarPasswordUpdate("newPass", "oldHash");
        test("U06", "Actualización de Contraseña (cambio)",
            "Verificando que al cambiar password se genere un nuevo hash diferente al anterior.",
            "\"newPass\", \"oldHash\"",
            !newHash.equals("oldHash") && !newHash.equals("newPass"), true);

        String keptHash = ut.procesarPasswordUpdate("", "oldHash");
        test("U07", "Pass Update (Vacío)",
            "Verificando que al dejar password vacío se mantenga el hash anterior.",
            "\"\", \"oldHash\"",
            keptHash.equals("oldHash"), true);

        public static void main(String[] args) {

            // U08: Modelo Usuario
            Usuario user = new Usuario("1", "Pepe", "Admin", "pepe@netfix.com", "passhash");
            boolean userOk = user.getIdUsuario().equals("1") && user.getNombre().equals("Pepe");
            test("U08", "Modelo Usuario Props",
                "Verificando constructor y getters de la clase Usuario.",
                "new Usuario(\"1\", \"Pepe\", \"Admin\", \"pepe@netfix.com\", \"passhash\")",
                userOk, true);

            // U09 - U10: Simulador
            SimuladorDiagnostico sim = new SimuladorDiagnostico();
            test("U09", "Simulador Instanciación",
                "Verificando que la clase SimuladorDiagnostico se instancie correctamente.",
                "new SimuladorDiagnostico()",
                sim != null, true);

            // Test extra de lógica simple
            test("U10", "Verificación Booleana Simple",
                "Prueba de control para verificar el runner.",
                "true",
                true, true);

            // --- PRUEBAS DE INTEGRACIÓN ---
            System.out.println("\n--- [2] PRUEBAS DE INTEGRACIÓN (DB) ---");

            // I01: Conexión DB
            boolean dbConnected = false;
            try {
                ResultSet rs = Utilities.ejecutarConsulta(Utilities.TipoConsulta.ROLES, 0);
                if (rs != null)
                    dbConnected = true;
            } catch (Exception e) {
                System.out.println("EXCEPTION en I01: " + e.getMessage());
            }
            test("I01", "Conexión DB (Consulta Roles)",
                "Intentando conectar a DB y ejecutar consulta SELECT de roles.",
                "TipoConsulta.ROLES, 0",
                dbConnected, true);

            // Resumen
            System.out.println("\n=====");
            System.out.println(" RESUMEN DE EJECUCIÓN");
            System.out.println("=====");
            System.out.println(" TOTAL : " + testsRun);
            System.out.println(" PASSED: " + testsPassed);
            System.out.println(" FAILED: " + testsFailed);
            System.out.println("=====");
        }
    }
}
```


2.5 Ejemplo de ejecución de la batería de pruebas:

```
private static void test(String id, String name, String description, String inputs, Object actual,
                        Object expected) {
    testsRun++;
    boolean passed = (actual == null && expected == null) || (actual != null && actual.equals(expected));

    System.out.println(String.format("[%s] %s", id, name));
    System.out.println("    Detalle: " + description);
    System.out.println("    Entradas: " + inputs);

    if (passed) {
        testsPassed++;
        System.out.println("        Resultado: [OK]");
    } else {
        testsFailed++;
        System.out.println("        Resultado: [FAIL]");
        System.out.println(String.format("        Esperado: %s", expected));
        System.out.println(String.format("        Obtenido: %s", actual));
    }
    System.out.println("-----");
}

}

=====
INICIANDO BATERÍA DE PRUEBAS NETFIX
=====

--- [1] PRUEBAS DE UNIDAD (LOGICA) ---
[U01] Email Valid (.com)
Detalle: Verificando que un email con dominio .com sea aceptado.
Entradas: "test@netfix.com"
Resultado: [OK]
-----
[U02] Email Valid (.es)
Detalle: Verificando que un email con dominio .es sea aceptado.
Entradas: "test@netfix.es"
Resultado: [OK]
-----
[U03] Email Invalid (gmail)
Detalle: Verificando que un email con dominio externo (gmail) sea rechazado.
Entradas: "test@gmail.com"
Resultado: [OK]
-----
[U04] Hashing no vacío
Detalle: Verificando que hashPass genere una cadena no vacía y diferente al input.
Entradas: "secret"
Resultado: [OK]
-----
[U05] BCrypt Verification
Detalle: Verificando que BCrypt valide correctamente el password original contra su hash.
Entradas: "secret", "$2a$10$hDk3th5Ljh2CmALBNdp.Xe43zcy2uledcvSVaTixnPelwbkmfI0Hsu"
Resultado: [OK]
-----
[U06] Pass Update (Cambio)
Detalle: Verificando que al cambiar password se genere un nuevo hash diferente al anterior.
Entradas: "newPass", "oldHash"
Resultado: [OK]
-----
[U07] Pass Update (Vacío)
Detalle: Verificando que al dejar password vacío se mantenga el hash anterior.
Entradas: "", "oldHash"
Resultado: [FAIL]
Esperado: true
Obtenido: false
-----
[U08] Modelo Usuario Props
Detalle: Verificando constructor y getters de la clase Usuario.
Entradas: new Usuario("1", "Pepe", "Admin", "pepe@netfix.com", "passhash")
Resultado: [OK]
-----
[U09] Simulador Instanciación
Detalle: Verificando que la clase SimuladorDiagnostico se instancie correctamente.
Entradas: new SimuladorDiagnostico()
Resultado: [OK]
-----
[U10] Verificación Booleana Simple
Detalle: Prueba de control para verificar el runner.
Entradas: true
Resultado: [OK]
-----

--- [2] PRUEBAS DE INTEGRACIÓN (DB) ---
[I01] Conexión DB (Consulta Roles)
Detalle: Intentando conectar a DB y ejecutar consulta SELECT de roles.
Entradas: TipoConsulta.ROLES, 0
Resultado: [OK]
-----

=====
```

3. Respuestas de Gestión y Planificación del Proyecto

Este documento recoge las respuestas a las cuestiones planteadas sobre la ejecución, legalidad y valoración económica del proyecto NETFIX.

3.1 ¿Ha podido cumplir la planificación? ¿Qué problemas ha tenido?

Sí, se ha cumplido la planificación general, aunque con desviaciones puntuales.

Se han completado todos los requisitos funcionales obligatorios (Login, Gestión de Usuarios, Panel de Administración, etc.) dentro del plazo. Sin embargo, la fase de desarrollo de la interfaz de usuario consumió aproximadamente un 20% más de tiempo del estimado inicialmente.

3.2 Principales problemas y ajustes:

- * **Curva de aprendizaje de FlatLaf:** Implementar un diseño moderno sobre Swing requirió más investigación de la prevista para personalizar componentes (bordes, colores, sombras).
- * **Gestión del Repositorio:** Hubo dificultades técnicas con Git (conflictos de ramas y problemas de codificación de caracteres) que obligaron a detener el desarrollo de nuevas "features" (como el modo oscuro) para dedicar tiempo a estabilizar y limpiar el código base.
- * **Adaptación de Herramientas:** La migración parcial a VSCode para editar la interfaz gráfica fue una decisión no planificada que, aunque beneficiosa a largo plazo, supuso un freno temporal en la velocidad de desarrollo durante los primeros días.

3.3 ¿Su proyecto necesita algún tipo de permiso o autorización administrativa?

No.

Al tratarse de un proyecto académico que simula un entorno empresarial con datos ficticios, no requiere licencias de actividad ni autorizaciones gubernamentales.

- * **Protección de Datos (RGPD):** Si el proyecto se desplegase en un entorno real con datos de clientes reales, sería obligatorio inscribir los ficheros ante la AEPD y cumplir con el RGPD. En este estado, al usar datos generados está exento.
- * **Propiedad Intelectual:** El código hace uso de librerías de código abierto (FlatLaf, JFreeChart, MySQL Connector) bajo licencias compatibles (Apache 2.0, LGPL, GPL), por lo que legalmente es viable su distribución cumpliendo con la atribución correspondiente.

3.4 ¿Ha establecido algún documento de prevención de riesgos laborales?

Dado que el desarrollo es una actividad de oficina/sedentaria, no existe un documento formal de PRL visado por un técnico, pero sí se han aplicado las siguientes pautas durante la ejecución:

- * **Ergonomía Visual:** Uso de monitores con protección de luz azul y configuración de la interfaz de desarrollo (IDE) en modo oscuro para reducir la fatiga visual tras jornadas largas.
- * **Higiene Postural:** Se han programado pausas activas cada 2 horas para estiramientos y descanso visual, utilizando una silla ergonómica con soporte lumbar adecuado.
- * **Entorno:** Ubicación del puesto de trabajo en zona con iluminación natural lateral para evitar reflejos en la pantalla.

3.5 Valoración económica respecto a su ejecución

Se estima un coste total de desarrollo de **3.150,00 €** si el proyecto se hubiese realizado en un entorno profesional junior.

Desglose de costes:

Horas de Desarrollo: 120 horas, 25,00 €/h, 3.000,00 €

Licencias Software: 0,00 €

Amortización Equipo: 150,00 € (parte prop.)

TOTAL: 3.150,00 €

Nota: Se han utilizado herramientas Open Source (NetBeans, VSCode, Git, MySQL Community), por lo que el coste de licencias es cero.

4. Guía de instalación/Configuración.

4.1 Configuración Base de datos.

Para el correcto funcionamiento de este proyecto, hay que importar la base de datos “netfix” al servidor MySQL. Para ello se adjunta en el repositorio de GITHUB el script SQL.

JMMUNOZ-DAM Script para la importación de la base de datos 5ddc784 · now 4 Commits		
.vscode	Commit para entrega	3 hours ago
docs	COMMIT PARA ENTREGA	1 hour ago
src/main	Commit para entrega	3 hours ago
.gitignore	Commit para entrega	3 hours ago
README.md	Commit para entrega	3 hours ago
TestUnIntegra_Docs.md	COMMIT PARA ENTREGA	1 hour ago
nb-configuration.xml	Commit para entrega	3 hours ago
nbactions.xml	Commit para entrega	3 hours ago
netfix.sql	Script para la importación de la base de datos	now
pom.xml	Commit para entrega	3 hours ago

Una vez ejecutado el Script, en PHPMyAdmin debe aparecer algo así:

The screenshot shows the PHPMyAdmin interface for a MySQL server. The left sidebar displays a tree view of databases, with 'netfix' selected. The main panel shows the 'Estructura' (Structure) tab for the 'netfix' database. A table with 13 rows lists the tables in the database, including 'agenda', 'aparatos', 'clientes', 'comentarios', 'contratos', 'diagnostico', 'horarios', 'incidencias', 'logs', 'numeros', 'roles', 'tecnicos', and 'usuarios'. Each row provides details such as the table name, actions (Examinar, Estructura, Buscar, Insertar, Vaciar, Eliminar), file size, and storage engine. The bottom of the table shows a summary: '13 tablas', 'Número de filas', and '1,124 InnoDB utf8mb4_general_ci 544.0 KB'.

Tabla	Acción	Filas	Tipo	Cotejamiento	Tamaño	Residuo a depurar
agenda	Examinar Estructura Buscar Insertar Vaciar Eliminar	8	InnoDB	utf8mb4_general_ci	48.0 KB	-
aparatos	Examinar Estructura Buscar Insertar Vaciar Eliminar	70	InnoDB	utf8mb4_general_ci	64.0 KB	-
clientes	Examinar Estructura Buscar Insertar Vaciar Eliminar	27	InnoDB	utf8mb4_general_ci	32.0 KB	-
comentarios	Examinar Estructura Buscar Insertar Vaciar Eliminar	22	InnoDB	utf8mb4_general_ci	16.0 KB	-
contratos	Examinar Estructura Buscar Insertar Vaciar Eliminar	49	InnoDB	utf8mb4_general_ci	48.0 KB	-
diagnostico	Examinar Estructura Buscar Insertar Vaciar Eliminar	17	InnoDB	utf8mb4_general_ci	32.0 KB	-
horarios	Examinar Estructura Buscar Insertar Vaciar Eliminar	60	InnoDB	utf8mb4_general_ci	32.0 KB	-
incidencias	Examinar Estructura Buscar Insertar Vaciar Eliminar	90	InnoDB	utf8mb4_general_ci	48.0 KB	-
logs	Examinar Estructura Buscar Insertar Vaciar Eliminar	725	InnoDB	utf8mb4_general_ci	96.0 KB	-
numeros	Examinar Estructura Buscar Insertar Vaciar Eliminar	32	InnoDB	utf8mb4_general_ci	48.0 KB	-
roles	Examinar Estructura Buscar Insertar Vaciar Eliminar	3	InnoDB	utf8mb4_general_ci	16.0 KB	-
tecnicos	Examinar Estructura Buscar Insertar Vaciar Eliminar	7	InnoDB	utf8mb4_general_ci	32.0 KB	-
usuarios	Examinar Estructura Buscar Insertar Vaciar Eliminar	14	InnoDB	utf8mb4_general_ci	32.0 KB	-
13 tablas Número de filas		1,124	InnoDB	utf8mb4_general_ci	544.0 KB	0 B

Entramos en cualquier tabla para observar que tenemos datos para poder continuar con la instalación del programa.

phpMyAdmin

Servidor: 127.0.0.1 » Base de datos: netfix » Tabla: aparatos

Examinar Estructura SQL Buscar Insertar Exportar Importar Privilegios Operaciones

Reciente Favoritas

Nueva
camping
empresa2324
information_schema
musica
mysql
netfix
Nueva
agenda
aparatos
clientes
comentarios
contratos
diagnostico
horarios
incidencias
logs
numeros
roles
tecnicos
usuarios
performance_schema
phpmyadmin
test
tierra

				id_aparato	tipo_aparato	marca	modelo	numero_serie	mac	id_contrato
☐	Editar	Copiar	Borrar	2001	FTTH	Huawei	HG8245	FTTH-2001	A1B2C3D4E5F1	72
☐	Editar	Copiar	Borrar	2002	FTTH	ZTE	F680	FTTH-2002	A1B2C3D4E5F2	1003
☐	Editar	Copiar	Borrar	2003	FTTH	Nokia	G-2425	FTTH-2003	A1B2C3D4E5F3	1005
☐	Editar	Copiar	Borrar	2004	FTTH	ZTE	F680	FTTH-2004	A1B2C3D4E5F4	1006
☐	Editar	Copiar	Borrar	2005	5G	Vodafone	SIM Voz	SIM-2005	000000000005	1002
☐	Editar	Copiar	Borrar	2006	5G	Movistar	SIM Datos	SIM-2006	000000000006	NULL
☐	Editar	Copiar	Borrar	2007	5G	Orange	SIM Voz	SIM-2007	000000000007	1003
☐	Editar	Copiar	Borrar	2008	5G	Yoigo	SIM Datos	SIM-2008	000000000008	1004
☐	Editar	Copiar	Borrar	2009	5G	Vodafone	SIM Voz	SIM-2009	000000000009	1006
☐	Editar	Copiar	Borrar	2010	5G	Vodafone	SIM Datos	SIM-2010	000000000010	1006
☐	Editar	Copiar	Borrar	2011	5G	Orange	SIM Voz	SIM-2011	000000000011	1007
☐	Editar	Copiar	Borrar	9201	FTTH	Huawei	HG8245	FTTH-9201	A1B2C3D4E501	9101
☐	Editar	Copiar	Borrar	9202	FTTH	ZTE	F680	FTTH-9202	A1B2C3D4E502	9103
☐	Editar	Copiar	Borrar	9203	FTTH	Nokia	G-2425	FTTH-9203	A1B2C3D4E503	9105
☐	Editar	Copiar	Borrar	9204	FTTH	ZTE	F680	FTTH-9204	A1B2C3D4E504	9106
☐	Editar	Copiar	Borrar	9205	FTTH	Huawei	HG8245	FTTH-9205	A1B2C3D4E505	9108
☐	Editar	Copiar	Borrar	9206	FTTH	Nokia	G-2425	FTTH-9206	A1B2C3D4E506	9109
☐	Editar	Copiar	Borrar	9207	FTTH	ZTE	F680	FTTH-9207	A1B2C3D4E507	9111
☐	Editar	Copiar	Borrar	9208	FTTH	Huawei	HG8245	FTTH-9208	A1B2C3D4E508	9112
☐	Editar	Copiar	Borrar	9209	FTTH	Nokia	G-2425	FTTH-9209	A1B2C3D4E509	9115
☐	Editar	Copiar	Borrar	9210	FTTH	ZTE	F680	FTTH-9210	A1B2C3D4E510	9116
☐	Editar	Copiar	Borrar	9211	FTTH	Huawei	HG8245	FTTH-9211	A1B2C3D4E511	9118
☐	Editar	Copiar	Borrar	9212	FTTH	Nokia	G-2425	FTTH-9212	A1B2C3D4E512	9121
☐	Editar	Copiar	Borrar	9213	FTTH	ZTE	F680	FTTH-9213	A1B2C3D4E513	9122
☐	Editar	Copiar	Borrar	9214	FTTH	Huawei	HG8245	FTTH-9214	A1B2C3D4E514	9124

El usuario para acceso a la base de datos debe ser:

Usuario: root

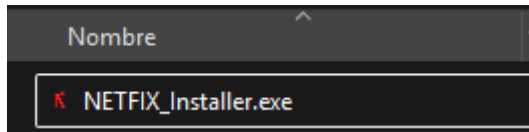
Contraseña: En blanco

Se puede consultar esto en la clase "DatabaseManager"

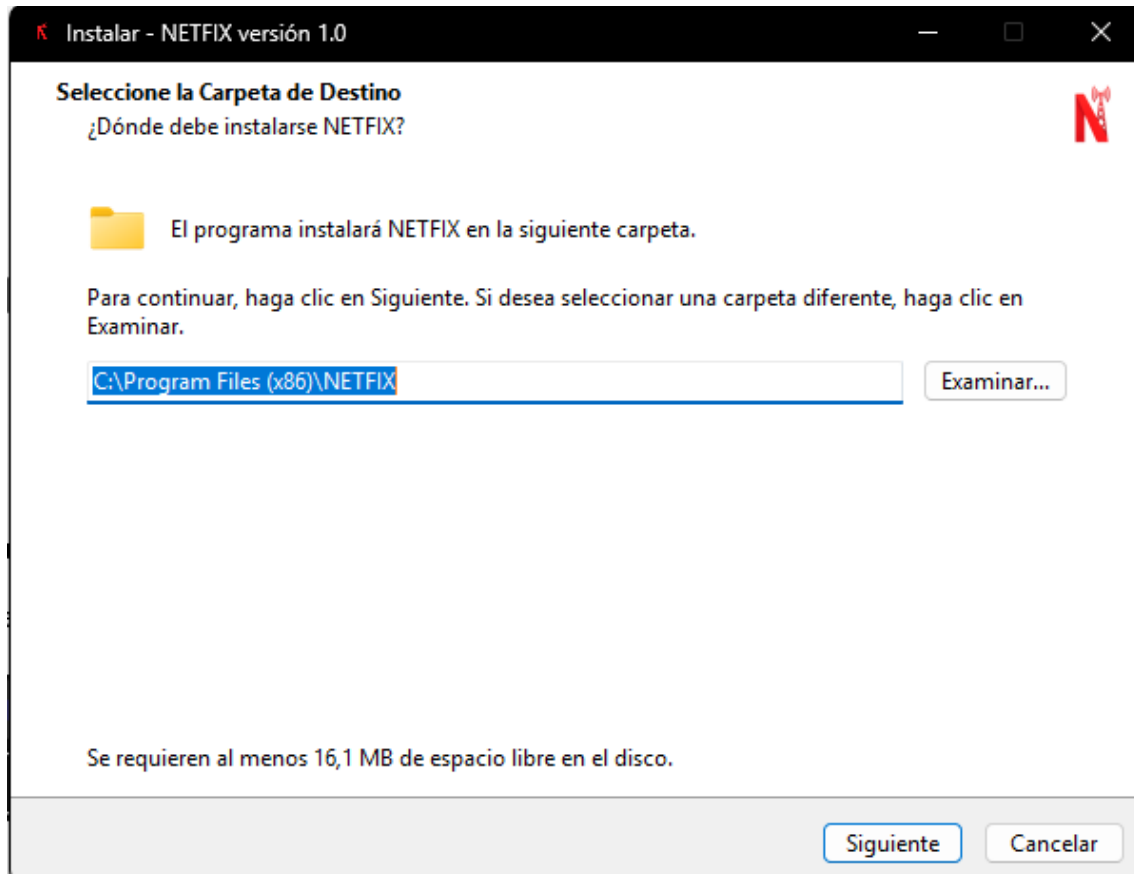
```
// =====
// CONFIGURACIÓN DE LA CONEXIÓN
// =====
private static final String URL = "jdbc:mysql://localhost:3306/netfix";
private static final String USER = "root";
private static final String PASSWORD = "";
```

4.2 Instalación de la aplicación.

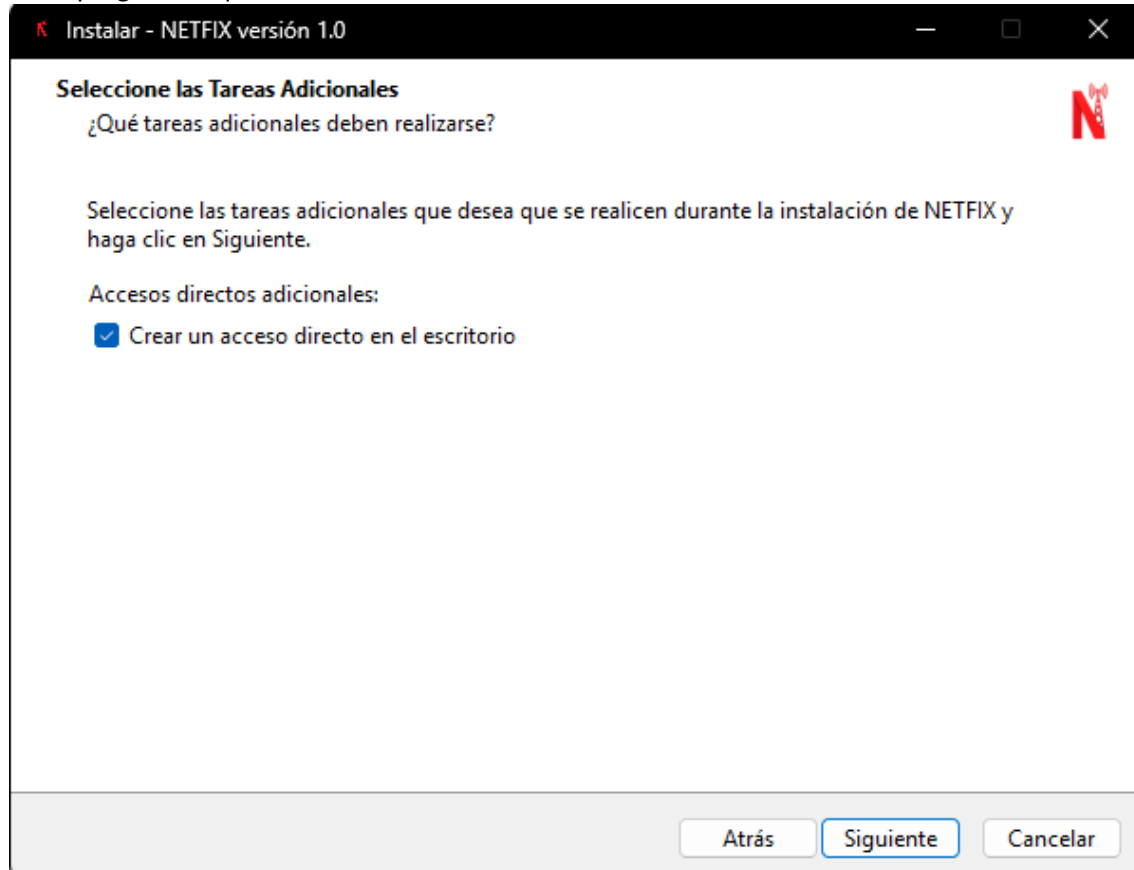
Primero hacemos doble clic en el instalador.



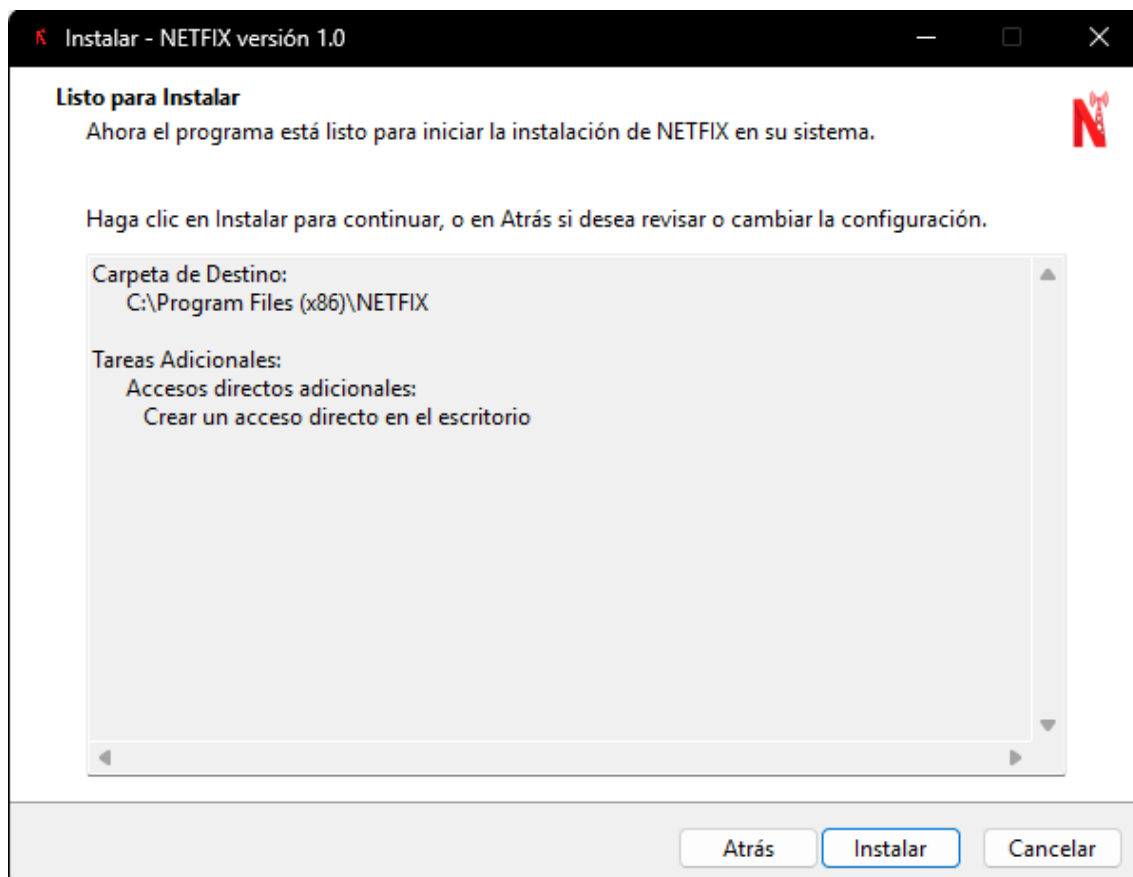
Una vez abierto, elegimos la carpeta donde se instalará:



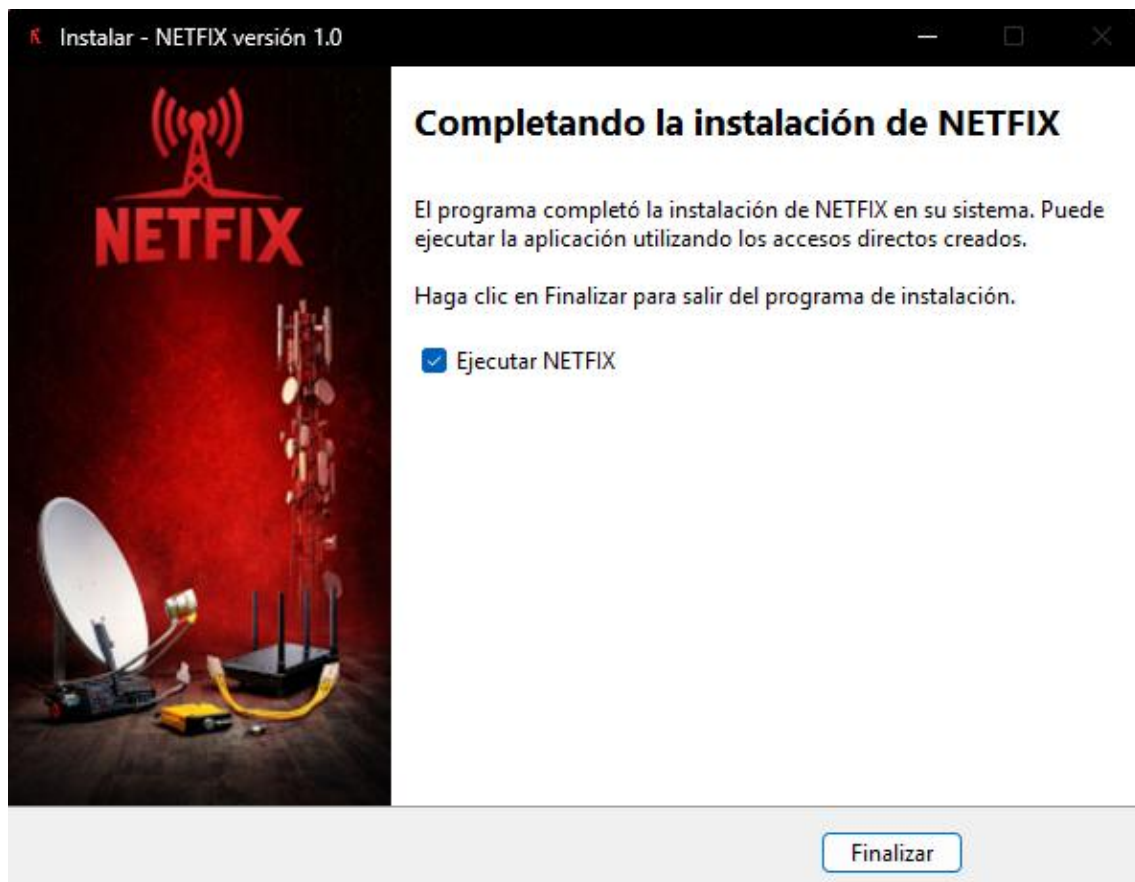
Nos pregunta si queremos crear un acceso directo.



Mostrará el resumen de lo seleccionado y procede con la instalación.



Una vez finalizado el proceso de instalación ya podemos ejecutar la aplicación.



5. Usuarios para realizar las pruebas.

Se han creado 3 usuarios con diferentes perfiles para poder probar la aplicación.

Contraseña global: prueba1234

Administrador: admin@netfix.es

Supervisor: lindsay@netfix.com

Técnico: ana.torres@netfix.com